程序鉴别材料（全文，共 2619 行，约 53 页）

```
// ===== FILE: packages/font-tool/src/policy.js =====
/**
 * Frozen whitespace / punctuation policy for glyph remapping.
 *
 * - Whitespace (JS `\s`, including space, tab, CR/LF, NBSP, ideographic space)
 *   is passed through unchanged so layout survives without a mapping.
 * - Every other visible character (letters, digits, CJK, punctuation) is
 *   mapped onto PUA starting at U+E000.
 * - Characters missing from the master font still get a PUA slot; the font
 *   builder paints a box (U+25A1 / .notdef) there.
 */

const PUA_BMP_START = 0xe000;
const PUA_BMP_END = 0xf8ff;
const PUA_SUPP_START = 0xf0000;

export function isPassthroughChar(ch) {
  return /\s/u.test(ch);
}

export function puaCodePoint(index) {
  const bmpSpan = PUA_BMP_END - PUA_BMP_START + 1;
  if (index < bmpSpan) {
    return PUA_BMP_START + index;
  }
  return PUA_SUPP_START + (index - bmpSpan);
}

export const TOFU_CODE_POINT = 0x25a1;
export const FONT_FAMILY_PREFIX = "pr-sess-";


// ===== FILE: packages/font-tool/src/mapText.js =====
import { isPassthroughChar, puaCodePoint } from "./policy.js";

/**
 * Build a stable char → PUA map for `text` and encode it.
 * Decode is for tests / server-side checks only — do not ship it in reader-web.
 */
export function buildMapping(text) {
  const forward = new Map();
  let nextIndex = 0;

  for (const ch of text) {
    if (isPassthroughChar(ch) || forward.has(ch)) {
      continue;
    }
    forward.set(ch, String.fromCodePoint(puaCodePoint(nextIndex)));
    nextIndex += 1;
  }
```

```
0051|
0052|   return { forward, uniqueCount: nextIndex };
0053| }
0054|
0055| export function encodeText(text, forward) {
0056|   let out = "";
0057|   for (const ch of text) {
0058|     out += isPassthroughChar(ch) ? ch : forward.get(ch);
0059|   }
0060|   return out;
0061| }
0062|
0063| export function decodeText(encoded, forward) {
0064|   const inverse = new Map();
0065|   for (const [from, to] of forward) {
0066|     inverse.set(to, from);
0067|   }
0068|   let out = "";
0069|   for (const ch of encoded) {
0070|     out += isPassthroughChar(ch) ? ch : inverse.get(ch) ?? ch;
0071|   }
0072|   return out;
0073| }
0074|
0075| export function mappingToJson(forward) {
0076|   const chars = {};
0077|   for (const [from, to] of forward) {
0078|     chars[from] = to;
0079|   }
0080|   return {
0081|     version: 1,
0082|     puaStart: "U+E000",
0083|     policy: {
0084|       passthrough: "javascript-unicode-whitespace",
0085|       punctuation: "mapped",
0086|       missingGlyph: "box",
0087|     },
0088|     chars,
0089|   };
0090| }
0091|
0092| export function mappingFromJson(json) {
0093|   const forward = new Map(Object.entries(json.chars));
0094|   return forward;
0095| }
0096|
0097|
0098| // ===== FILE: packages/font-tool/src/buildFont.js =====
0099| import { spawn } from "node:child_process";
0100| import fs from "node:fs";
```

```
import path from "node:path";
import { fileURLToPath } from "node:url";

const here = path.dirname(fileURLToPath(import.meta.url));
const SCRIPT = path.resolve(here, "../python/remap_subset.py");

export function findMasterFont(fontsMasterDir) {
  const preferred = [
    "NotoSansSC-Regular.otf",
    "NotoSansSC-Regular.ttf",
    "NotoSansSC-Regular.woff",
    "NotoSansSC-Regular.woff2",
  ];
  for (const name of preferred) {
    const full = path.join(fontsMasterDir, name);
    if (fs.existsSync(full) && fs.statSync(full).size > 100_000) {
      return full;
    }
  }
  return null;
}

function pythonBin() {
  return process.env.READER_PYTHON || process.env.PYTHON || "python3";
}

export function buildWoff2(job) {
  return new Promise((resolve, reject) => {
    const child = spawn(pythonBin(), [SCRIPT], {
      stdio: ["pipe", "pipe", "pipe"],
    });
    let stderr = "";
    child.stderr.on("data", (chunk) => {
      stderr += chunk.toString();
    });
    child.on("error", reject);
    child.on("close", (code) => {
      if (code === 0) {
        resolve();
      } else {
        reject(new Error(`remap_subset.py exited ${code}: ${stderr}`));
      }
    });
    child.stdin.write(JSON.stringify(job));
    child.stdin.end();
  });
}


// ===== FILE: packages/font-tool/src/cli.js =====
```

```
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { buildWoff2, findMasterFont } from "./buildFont.js";
import {
  buildMapping,
  encodeText,
  mappingToJson,
} from "./mapText.js";
import { FONT_FAMILY_PREFIX } from "./policy.js";

const here = path.dirname(fileURLToPath(import.meta.url));
const repoRoot = path.resolve(here, "../../..");

function htmlEscape(text) {
  return text
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;");
}

function buildPreviewHtml({ fontFamily, encoded, fontUrl }) {
  return `<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8" />
  <title>petrichor-reader font preview</title>
  <style>
    @font-face {
      font-family: "${fontFamily}";
      src: url("${fontUrl}") format("woff2");
      font-display: block;
    }
    body {
      margin: 2rem auto;
      max-width: 40rem;
      line-height: 1.9;
      font-size: 20px;
      color: #1a1a1a;
      background: #f6f1e7;
    }
    .hint { font-family: sans-serif; font-size: 14px; color: #555; }
    .chapter {
      font-family: "${fontFamily}", serif;
      white-space: pre-wrap;
      word-break: break-word;
    }
  </style>
</head>
<body>
```

```
  <p class="hint">人眼应能读正文。查看源代码应只有 PUA 码位，没有原句。禁用此自定义字体后应变乱码/方框。</p>
  <div class="chapter">${htmlEscape(encoded)}</div>
</body>
</html>
`;
}

async function main() {
  const chapterPath = path.join(repoRoot, "fixtures/chapters/sample.txt");
  const outputDir = path.join(here, "../output");
  const fixturesDir = path.join(repoRoot, "fixtures");
  const fontsMasterDir = path.join(repoRoot, "fonts/master");
  const masterPath = findMasterFont(fontsMasterDir);
  if (!masterPath) {
    throw new Error(
      "Missing master font. Run: node packages/font-tool/scripts/download-master-font.mjs",
    );
  }

  const text = fs.readFileSync(chapterPath, "utf8").replace(/^\uFEFF/, "");
  const { forward } = buildMapping(text);
  const encoded = encodeText(text, forward);
  const fontFamily = `${FONT_FAMILY_PREFIX}preview`;
  const psName = "PrSessPreview";

  fs.mkdirSync(outputDir, { recursive: true });
  const woff2Path = path.join(outputDir, "subset.woff2");
  const mapPath = path.join(outputDir, "map.json");

  const mappings = [];
  for (const [from, to] of forward) {
    mappings.push({
      from: from.codePointAt(0),
      to: to.codePointAt(0),
    });
  }

  await buildWoff2({
    masterPath,
    outputPath: woff2Path,
    fontFamily,
    psName,
    mappings,
    passthrough: [9, 10, 13, 32, 160, 0x3000],
  });

  fs.writeFileSync(mapPath, JSON.stringify(mappingToJson(forward), null, 2), "utf8");

  const previewAssets = path.join(fixturesDir, "preview-assets");
  fs.mkdirSync(previewAssets, { recursive: true });
```

```
  fs.copyFileSync(woff2Path, path.join(previewAssets, "subset.woff2"));

  const previewHtml = buildPreviewHtml({
    fontFamily,
    encoded,
    fontUrl: "./preview-assets/subset.woff2",
  });
  fs.writeFileSync(path.join(fixturesDir, "preview.html"), previewHtml, "utf8");

  const originalSnippet = text.trim().split(/\n/)[0];
  if (encoded.includes(originalSnippet) || previewHtml.includes(originalSnippet)) {
    throw new Error("Encoded preview still contains the original first line.");
  }

  console.log(`master: ${masterPath}`);
  console.log(`woff2:  ${woff2Path}`);
  console.log(`map:    ${mapPath} (tool-internal; do not ship to reader-web)`);
  console.log(`preview: ${path.join(fixturesDir, "preview.html")}`);
}

main().catch((err) => {
  console.error(err);
  process.exit(1);
});


// ===== FILE: packages/font-tool/python/remap_subset.py =====
"""Subset a master font and remap listed glyphs onto PUA codepoints."""

import json
import sys

from fontTools.subset import Options, Subsetter
from fontTools.ttLib import TTFont
from fontTools.ttLib.tables._c_m_a_p import CmapSubtable

TOFU = 0x25A1


def _set_family_names(font, family, ps_name):
    name = font["name"]
    pairs = [
        (1, family),
        (2, "Regular"),
        (3, "%s;petrichor-reader" % ps_name),
        (4, "%s Regular" % family),
        (6, ps_name),
        (16, family),
        (17, "Regular"),
    ]
```

```
    for name_id, value in pairs:
        written = False
        for plat_id, enc_id, lang_id in ((3, 1, 0x409), (1, 0, 0)):
            try:
                name.setName(value, name_id, plat_id, enc_id, lang_id)
                written = True
            except Exception:
                continue
        if not written:
            name.setName(value, name_id, 3, 1, 0x409)


def _replace_cmap(font, mapping):
    table = font["cmap"]
    max_cp = max(mapping) if mapping else 0
    subtables = []

    def add(fmt, plat, enc):
        sub = CmapSubtable.newSubtable(fmt)
        sub.platformID = plat
        sub.platEncID = enc
        sub.language = 0
        sub.cmap = dict(mapping)
        subtables.append(sub)

    if max_cp > 0xFFFF:
        add(12, 3, 10)
        add(12, 0, 4)
    else:
        add(4, 3, 1)
        add(4, 0, 3)

    table.tables = subtables


def build(job):
    font = TTFont(job["masterPath"], lazy=False)
    original_cmap = font.getBestCmap() or {}

    passthrough = set(int(cp) for cp in job.get("passthrough", []))
    passthrough.add(0x20)

    needed = set(passthrough)
    needed.add(TOFU)

    glyph_from = {}
    for item in job["mappings"]:
        src = int(item["from"])
        if src in original_cmap:
            glyph_from[src] = original_cmap[src]
```

```
            needed.add(src)
        else:
            glyph_from[src] = None
            needed.add(TOFU)

    options = Options()
    options.desubroutinize = True
    options.hinting = False
    options.layout_features = []
    options.name_IDs = ["*"]
    options.notdef_outline = True
    options.recommended_glyphs = True
    drop = getattr(options, "drop_tables", None)
    if isinstance(drop, set):
        drop.add("DSIG")
    elif drop is not None:
        options.drop_tables = list(drop) + ["DSIG"]

    subsetter = Subsetter(options=options)
    subsetter.populate(unicodes=needed)
    subsetter.subset(font)

    cmap = font.getBestCmap() or {}
    tofu_name = cmap.get(TOFU, ".notdef")
    order = font.getGlyphOrder()

    new_map = {}
    for cp in passthrough:
        if cp in cmap:
            new_map[cp] = cmap[cp]

    for item in job["mappings"]:
        src = int(item["from"])
        dst = int(item["to"])
        gname = glyph_from[src]
        if gname is None:
            new_map[dst] = tofu_name
        elif src in cmap:
            new_map[dst] = cmap[src]
        elif gname in order:
            new_map[dst] = gname
        else:
            new_map[dst] = tofu_name

    _replace_cmap(font, new_map)
    _set_family_names(font, job["fontFamily"], job["psName"])

    font.flavor = "woff2"
    font.save(job["outputPath"])
    font.close()
```

```
0401|
0402|
0403| def main():
0404|     job = json.load(sys.stdin)
0405|     build(job)
0406|
0407|
0408| if __name__ == "__main__":
0409|     main()
0410|
0411|
0412| // ===== FILE: packages/font-tool/scripts/download-master-font.mjs =====
0413| /**
0414|  * Download Noto Sans SC Regular (static) into fonts/master/.
0415|  * Prefers the official regional-subset OTF; falls back to Fontsource static woff.
0416|  */
0417| import { spawn } from "node:child_process";
0418| import fs from "node:fs";
0419| import path from "node:path";
0420| import { fileURLToPath } from "node:url";
0421|
0422| const here = path.dirname(fileURLToPath(import.meta.url));
0423| const repoRoot = path.resolve(here, "../../..");
0424| const masterDir = path.join(repoRoot, "fonts/master");
0425| const oflPath = path.join(repoRoot, "fonts/OFL.txt");
0426|
0427| const OTF_URLS = [
0428|   "https://github.com/notofonts/noto-cjk/raw/main/Sans/SubsetOTF/SC/NotoSansSC-Regular.otf",
0429|   "https://ghproxy.net/https://github.com/notofonts/noto-cjk/raw/main/Sans/SubsetOTF/SC/NotoSansSC-R
0430| egular.otf",
0431|   "https://gh-proxy.com/https://github.com/notofonts/noto-cjk/raw/main/Sans/SubsetOTF/SC/NotoSansSC-
0432| Regular.otf",
0433| ];
0434|
0435| const WOFF_URLS = [
0436|   "https://cdn.jsdelivr.net/npm/@fontsource/noto-sans-sc@5.2.8/files/noto-sans-sc-chinese-simplified
0437| -400-normal.woff",
0438| ];
0439|
0440| const OFL_URLS = [
0441|   "https://cdn.jsdelivr.net/npm/@fontsource/noto-sans-sc@5.2.8/LICENSE",
0442| ];
0443|
0444| function curlTo(url, dest, extraArgs = []) {
0445|   return new Promise((resolve, reject) => {
0446|     const args = ["-L", "--fail", "--connect-timeout", "20", "--max-time", "300", ...extraArgs, "-o"
0447| , dest, url];
0448|     const child = spawn("curl.exe", args, { stdio: "inherit" });
0449|     child.on("error", reject);
0450|     child.on("close", (code) => {
```

```
      if (code === 0) resolve();
      else reject(new Error(`curl ${code} for ${url}`));
    });
  });
}

async function tryUrls(urls, dest, extraArgs = []) {
  fs.mkdirSync(path.dirname(dest), { recursive: true });
  let lastErr;
  for (const url of urls) {
    try {
      console.log(`GET ${url}`);
      await curlTo(url, dest, extraArgs);
      const size = fs.statSync(dest).size;
      if (size < 50_000) {
        throw new Error(`file too small: ${size}`);
      }
      console.log(`saved ${dest} (${size} bytes)`);
      return dest;
    } catch (err) {
      lastErr = err;
      console.warn(String(err.message || err));
    }
  }
  throw lastErr ?? new Error("no urls");
}

async function main() {
  fs.mkdirSync(masterDir, { recursive: true });
  const otfDest = path.join(masterDir, "NotoSansSC-Regular.otf");
  try {
    const resume = fs.existsSync(otfDest) ? ["-C", "-"] : [];
    await tryUrls(OTF_URLS, otfDest, resume);
  } catch {
    console.warn("Official OTF unavailable; falling back to Fontsource static woff.");
    await tryUrls(WOFF_URLS, path.join(masterDir, "NotoSansSC-Regular.woff"));
  }

  if (!fs.existsSync(oflPath) || fs.statSync(oflPath).size < 100) {
    await tryUrls(OFL_URLS, oflPath);
  }
}

main().catch((err) => {
  console.error(err);
  process.exit(1);
});


// ===== FILE: packages/reader-api/src/config.js =====
```

```
export const DEFAULT_PORT = 3980;
export const DEFAULT_PUBLIC_BASE = "http://localhost:3980";
export const DEFAULT_ALLOWED_ORIGINS = [
  "http://localhost:3981",
  "http://127.0.0.1:3981",
];
export const DEFAULT_SITE_KEY = "pk_dev";
export const DEFAULT_TICKET_TTL_MS = 5 * 60 * 1000;
export const HAN_RE = /[\u3400-\u9fff]/u;

export function containsHan(text) {
  return HAN_RE.test(text);
}

export function loadConfig(overrides = {}) {
  return {
    port: Number(overrides.port ?? process.env.READER_API_PORT ?? DEFAULT_PORT),
    publicBase: String(
      overrides.publicBase ?? process.env.READER_API_PUBLIC_BASE ?? DEFAULT_PUBLIC_BASE,
    ).replace(/\/$/, ""),
    allowedOrigins: overrides.allowedOrigins ?? (
      process.env.READER_ALLOWED_ORIGINS
        ? process.env.READER_ALLOWED_ORIGINS.split(",").map((s) => s.trim()).filter(Boolean)
        : DEFAULT_ALLOWED_ORIGINS
    ),
    siteKey: overrides.siteKey ?? process.env.READER_SITE_KEY ?? DEFAULT_SITE_KEY,
    ticketTtlMs: Number(
      overrides.ticketTtlMs ?? process.env.READER_TICKET_TTL_MS ?? DEFAULT_TICKET_TTL_MS,
    ),
    rateLimitPerMin: Number(overrides.rateLimitPerMin ?? 240),
    articleByIdUrl: overrides.articleByIdUrl ?? process.env.READER_ARTICLE_BY_ID_URL ?? "",
    repoRoot: overrides.repoRoot,
  };
}


// ===== FILE: packages/reader-api/src/revision.js =====
import crypto from "node:crypto";

export function normalizeUpdatedAt(value) {
  if (value == null || value === "") {
    return "";
  }
  return String(value);
}

export function contentRevision(title, body) {
  return crypto.createHash("sha1").update(`${title || ""}\n${body || ""}`).digest("hex").slice(0, 16
);
}
```

```
0551|
0552| export function chapterRevision(chapter) {
0553|   const updatedAt = normalizeUpdatedAt(chapter?.updatedAt);
0554|   if (updatedAt) {
0555|     return updatedAt;
0556|   }
0557|   return contentRevision(chapter?.title, chapter?.body);
0558| }
0559|
0560| export function fontCacheKey(chapter) {
0561|   return `${chapter.id}\x1e${chapterRevision(chapter)}`;
0562| }
0563|
0564| export function evictOtherRevisions(cache, chapterId, keepKey) {
0565|   const prefix = `${chapterId}\x1e`;
0566|   for (const key of cache.keys()) {
0567|     if (key !== keepKey && key.startsWith(prefix)) {
0568|       cache.delete(key);
0569|     }
0570|   }
0571| }
0572|
0573|
0574| // ===== FILE: packages/reader-api/src/richText.js =====
0575| import { encodeText } from "../../font-tool/src/mapText.js";
0576|
0577| const LOOKS_HTML = /<\/?[a-zA-Z]/;
0578| const VOID = new Set(["br", "img", "hr"]);
0579| const ALLOWED = new Set([
0580|   "p",
0581|   "br",
0582|   "strong",
0583|   "b",
0584|   "em",
0585|   "i",
0586|   "u",
0587|   "blockquote",
0588|   "h1",
0589|   "h2",
0590|   "h3",
0591|   "ul",
0592|   "ol",
0593|   "li",
0594|   "a",
0595|   "img",
0596| ]);
0597| const UNWRAP = new Set(["span", "div", "font", "section", "article"]);
0598| const DROP = new Set(["script", "style", "iframe", "object", "embed", "link", "meta", "noscript"]);
0599| const ALIGN = {
0600|   "ql-align-left": "pr-align-left",
```

```
  "ql-align-center": "pr-align-center",
  "ql-align-right": "pr-align-right",
  "ql-align-justify": "pr-align-justify",
};
const ENTITIES = {
  nbsp: "\u00a0",
  amp: "&",
  lt: "<",
  gt: ">",
  quot: '"',
  apos: "'",
};

export function decodeEntities(text) {
  return String(text).replace(/&(#x?[0-9a-f]+|[a-z]+);/gi, (match, ent) => {
    const key = ent.toLowerCase();
    if (ENTITIES[key]) {
      return ENTITIES[key];
    }
    if (key.startsWith("#x")) {
      return String.fromCodePoint(Number.parseInt(key.slice(2), 16) || 32);
    }
    if (key.startsWith("#")) {
      return String.fromCodePoint(Number.parseInt(key.slice(1), 10) || 32);
    }
    return match;
  });
}

function escapeText(text) {
  return String(text)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;");
}

function escapeAttr(text) {
  return escapeText(text).replaceAll('"', "&quot;");
}

function plainToHtml(text) {
  const parts = String(text).replace(/\r\n/g, "\n").split(/\n{2,}/);
  return parts
    .map((para) => `<p>${escapeText(para).replaceAll("\n", "<br>")}</p>`)
    .join("");
}

function safeHref(url) {
  const t = String(url || "").trim();
  if (/^https?:\/\//i.test(t) || /^mailto:/i.test(t)) {
```

```
    return t;
  }
  if (t.startsWith("/") && !t.startsWith("//")) {
    return t;
  }
  return "";
}

function safeSrc(url) {
  const t = String(url || "").trim();
  if (/^https?:\/\//i.test(t)) {
    return t;
  }
  if (t.startsWith("/") && !t.startsWith("//")) {
    return t;
  }
  return "";
}

function parseAttrs(raw) {
  const attrs = {};
  const re = /([^\s=]+)(?:\s*=\s*(?:"([^"]*)"|'([^']*)'|([^\s"'>]+)))?/g;
  let m;
  while ((m = re.exec(raw))) {
    attrs[m[1].toLowerCase()] = decodeEntities(m[2] ?? m[3] ?? m[4] ?? "");
  }
  return attrs;
}

function skipUntil(html, i, needle) {
  const at = html.toLowerCase().indexOf(needle, i);
  return at === -1 ? html.length : at + needle.length;
}

function parseFragment(html) {
  const root = { type: "root", children: [] };
  const stack = [root];
  let i = 0;
  const src = String(html);

  function parent() {
    return stack[stack.length - 1];
  }

  function pushText(value) {
    if (!value) {
      return;
    }
    const kids = parent().children;
    const last = kids[kids.length - 1];
```

```
    if (last && last.type === "text") {
      last.value += value;
      return;
    }
    kids.push({ type: "text", value });
  }

  while (i < src.length) {
    if (src.startsWith("<!--", i)) {
      i = skipUntil(src, i + 4, "-->");
      continue;
    }
    if (src[i] !== "<") {
      const next = src.indexOf("<", i);
      const raw = next === -1 ? src.slice(i) : src.slice(i, next);
      pushText(decodeEntities(raw));
      i = next === -1 ? src.length : next;
      continue;
    }

    const close = src.indexOf(">", i + 1);
    if (close === -1) {
      pushText(decodeEntities(src.slice(i)));
      break;
    }
    const rawTag = src.slice(i + 1, close);
    i = close + 1;
    if (!rawTag || rawTag[0] === "!" || rawTag[0] === "?") {
      continue;
    }

    const selfClose = rawTag.endsWith("/");
    const body = selfClose ? rawTag.slice(0, -1).trim() : rawTag.trim();
    const isClose = body[0] === "/";
    const parts = (isClose ? body.slice(1) : body).match(/^([a-zA-Z][a-zA-Z0-9]*)\s*([\s\S]*)$/);
    if (!parts) {
      continue;
    }
    const tag = parts[1].toLowerCase();
    const attrRaw = parts[2] || "";

    if (isClose) {
      for (let s = stack.length - 1; s > 0; s -= 1) {
        if (stack[s].tag === tag) {
          stack.length = s;
          break;
        }
      }
      continue;
    }
```

```
0751|
0752|     if (DROP.has(tag)) {
0753|       if (!VOID.has(tag) && !selfClose) {
0754|         i = skipUntil(src, i, `</${tag}`);
0755|         const end = src.indexOf(">", i);
0756|         i = end === -1 ? src.length : end + 1;
0757|       }
0758|       continue;
0759|     }
0760|
0761|     if (UNWRAP.has(tag)) {
0762|       continue;
0763|     }
0764|
0765|     if (!ALLOWED.has(tag)) {
0766|       continue;
0767|     }
0768|
0769|     const node = { type: "element", tag, attrs: {}, children: [] };
0770|     const parsed = parseAttrs(attrRaw);
0771|     if (tag === "img") {
0772|       const href = safeSrc(parsed.src);
0773|       if (!href) {
0774|         continue;
0775|       }
0776|       node.attrs.src = href;
0777|       if (parsed.alt) {
0778|         node.attrs.alt = parsed.alt;
0779|       }
0780|     } else if (tag === "a") {
0781|       const href = safeHref(parsed.href);
0782|       if (href) {
0783|         node.attrs.href = href;
0784|         node.attrs.rel = "noopener noreferrer";
0785|       }
0786|     }
0787|     if (parsed.class) {
0788|       for (const cls of String(parsed.class).split(/\s+/)) {
0789|         if (ALIGN[cls]) {
0790|           node.attrs.class = ALIGN[cls];
0791|           break;
0792|         }
0793|       }
0794|     }
0795|
0796|     parent().children.push(node);
0797|     if (!VOID.has(tag) && !selfClose) {
0798|       stack.push(node);
0799|     }
0800|   }
```

```
0801|
0802|   return root;
0803| }
0804|
0805| function walk(node, visit) {
0806|   if (node.type === "text") {
0807|     visit(node);
0808|     return;
0809|   }
0810|   if (node.attrs?.alt) {
0811|     visit({ type: "attr", key: "alt", node });
0812|   }
0813|   if (node.attrs?.title) {
0814|     visit({ type: "attr", key: "title", node });
0815|   }
0816|   for (const child of node.children || []) {
0817|     walk(child, visit);
0818|   }
0819| }
0820|
0821| export function collectVisibleText(html) {
0822|   let out = "";
0823|   walk(parseFragment(html), (item) => {
0824|     if (item.type === "text") {
0825|       out += item.value;
0826|     } else if (item.type === "attr") {
0827|       out += item.node.attrs[item.key] || "";
0828|     }
0829|   });
0830|   return out;
0831| }
0832|
0833| function serialize(node) {
0834|   if (node.type === "text") {
0835|     return escapeText(node.value);
0836|   }
0837|   if (node.type === "root") {
0838|     return (node.children || []).map(serialize).join("");
0839|   }
0840|   const attrs = Object.entries(node.attrs || {})
0841|     .map(([k, v]) => ` ${k}="${escapeAttr(v)}"`)
0842|     .join("");
0843|   if (VOID.has(node.tag)) {
0844|     return `<${node.tag}${attrs}>`;
0845|   }
0846|   const inner = (node.children || []).map(serialize).join("");
0847|   if (node.tag === "a" && !node.attrs.href) {
0848|     return inner;
0849|   }
0850|   return `<${node.tag}${attrs}>${inner}</${node.tag}>`;
```

```
}

export function normalizeChapterBody(raw) {
  const s = String(raw || "").trim();
  if (!s) {
    return "";
  }
  const html = LOOKS_HTML.test(s) ? s : plainToHtml(s);
  return serialize(parseFragment(html));
}

export function chapterBodyHasContent(html) {
  if (collectVisibleText(html).trim()) {
    return true;
  }
  return /<img\b/i.test(html);
}

export function encodeRichHtml(html, forward) {
  const root = parseFragment(html);
  walk(root, (item) => {
    if (item.type === "text") {
      item.value = encodeText(item.value, forward);
    } else if (item.type === "attr") {
      item.node.attrs[item.key] = encodeText(item.node.attrs[item.key] || "", forward);
    }
  });
  return serialize(root);
}


// ===== FILE: packages/reader-api/src/chapters.js =====
import fs from "node:fs";
import path from "node:path";

export function loadChapters(repoRoot) {
  const dir = path.join(repoRoot, "fixtures/chapters");
  const indexPath = path.join(dir, "index.json");
  const index = JSON.parse(fs.readFileSync(indexPath, "utf8"));
  const byId = new Map();

  for (const entry of index.chapters) {
    const filePath = path.join(dir, entry.file);
    const body = fs.readFileSync(filePath, "utf8").replace(/^\uFEFF/, "");
    const updatedAt = String(fs.statSync(filePath).mtimeMs);
    byId.set(entry.id, {
      id: entry.id,
      title: entry.title,
      body,
      updatedAt,
```

```
    });
  }

  return byId;
}


// ===== FILE: packages/reader-api/src/rateLimit.js =====
export function createRateLimiter({ perMin }) {
  const windows = new Map();

  return function check(ip) {
    const now = Date.now();
    const key = ip || "unknown";
    let slot = windows.get(key);
    if (!slot || now - slot.start >= 60_000) {
      slot = { start: now, count: 0 };
      windows.set(key, slot);
    }
    slot.count += 1;
    return slot.count <= perMin;
  };
}

export function clientIp(req) {
  const forwarded = req.headers["x-forwarded-for"];
  if (typeof forwarded === "string" && forwarded.trim()) {
    return forwarded.split(",")[0].trim();
  }
  return req.socket.remoteAddress || "";
}


// ===== FILE: packages/reader-api/src/tickets.js =====
import crypto from "node:crypto";

export function createTicketStore({ ttlMs, now = () => Date.now() }) {
  const tickets = new Map();
  const fonts = new Map();

  function prune() {
    const t = now();
    for (const [id, row] of tickets) {
      if (row.expiresAt <= t) {
        tickets.delete(id);
      }
    }
    for (const [id, row] of fonts) {
      if (row.expiresAt <= t) {
        fonts.delete(id);
```

```
      }
    }
  }

  function issue(chapterId, sessionId) {
    prune();
    const ticket = crypto.randomBytes(24).toString("base64url");
    const fontId = crypto.randomBytes(16).toString("hex");
    const expiresAt = now() + ttlMs;
    tickets.set(ticket, {
      ticket,
      chapterId,
      sessionId,
      fontId,
      expiresAt,
    });
    return tickets.get(ticket);
  }

  function get(ticket) {
    prune();
    const row = tickets.get(ticket);
    if (!row || row.expiresAt <= now()) {
      tickets.delete(ticket);
      return null;
    }
    return row;
  }

  function putFont(fontId, record) {
    fonts.set(fontId, record);
  }

  function getFont(fontId) {
    prune();
    const row = fonts.get(fontId);
    if (!row || row.expiresAt <= now()) {
      fonts.delete(fontId);
      return null;
    }
    return row;
  }

  return { issue, get, putFont, getFont, prune };
}


// ===== FILE: packages/reader-api/src/remoteArticle.js =====
import { chapterBodyHasContent, collectVisibleText, normalizeChapterBody } from "./richText.js";
import { normalizeUpdatedAt } from "./revision.js";
```

```
1001|
1002| /** @deprecated kept for tests that only need visible text */
1003| export function htmlToPlain(html) {
1004|   return collectVisibleText(normalizeChapterBody(html)).replace(/\u00a0/g, " ").trim();
1005| }
1006|
1007| export async function fetchArticleById(url, id) {
1008|   const res = await fetch(url, {
1009|     method: "POST",
1010|     headers: { "Content-Type": "application/x-www-form-urlencoded" },
1011|     body: new URLSearchParams({ id }),
1012|   });
1013|   if (!res.ok) {
1014|     return null;
1015|   }
1016|   const json = await res.json();
1017|   const article = json && json.data;
1018|   if (!article) {
1019|     return null;
1020|   }
1021|   const articleId = String(article.id || id);
1022|   const title = [article.articleChapter, article.articleTitle]
1023|     .filter(Boolean)
1024|     .join(" - ");
1025|   const body = normalizeChapterBody(
1026|     [article.articleHead, article.articleContent, article.articleTail]
1027|       .filter(Boolean)
1028|       .join(""),
1029|   );
1030|   if (!chapterBodyHasContent(body)) {
1031|     return null;
1032|   }
1033|   return {
1034|     id: articleId,
1035|     title: title || articleId,
1036|     body,
1037|     updatedAt: normalizeUpdatedAt(article.updateTime || article.createTime || ""),
1038|   };
1039| }
1040|
1041|
1042| // ===== FILE: packages/reader-api/src/obfuscate.js =====
1043| import fs from "node:fs";
1044| import path from "node:path";
1045| import { buildMapping, encodeText } from "../../font-tool/src/mapText.js";
1046| import { buildWoff2, findMasterFont } from "../../font-tool/src/buildFont.js";
1047| import { FONT_FAMILY_PREFIX } from "../../font-tool/src/policy.js";
1048| import { containsHan } from "./config.js";
1049| import { evictOtherRevisions, fontCacheKey } from "./revision.js";
1050| import { collectVisibleText, encodeRichHtml, normalizeChapterBody } from "./richText.js";
```

```
1051|
1052| export function createObfuscator({
1053|   repoRoot,
1054|   generatedDir,
1055|   masterPath: masterPathOverride,
1056|   buildFont = buildWoff2,
1057| } = {}) {
1058|   let masterPath = masterPathOverride ?? null;
1059|   fs.mkdirSync(generatedDir, { recursive: true });
1060|   const cache = new Map();
1061|
1062|   function resolveMaster() {
1063|     if (masterPath) {
1064|       return masterPath;
1065|     }
1066|     masterPath = findMasterFont(path.join(repoRoot, "fonts/master"));
1067|     if (!masterPath) {
1068|       throw new Error("master font missing");
1069|     }
1070|     return masterPath;
1071|   }
1072|
1073|   async function forChapter(chapter) {
1074|     const key = fontCacheKey(chapter);
1075|     const hit = cache.get(key);
1076|     if (hit) {
1077|       return hit;
1078|     }
1079|
1080|     const title = chapter.title || "";
1081|     const html = normalizeChapterBody(chapter.body);
1082|     const visible = collectVisibleText(html);
1083|     const combined = title ? `${title}\n${visible}` : visible;
1084|     const { forward } = buildMapping(combined);
1085|     const titleGlyphs = title ? encodeText(title, forward) : "";
1086|     const bodyGlyphs = encodeRichHtml(html, forward);
1087|
1088|     if (containsHan(collectVisibleText(bodyGlyphs)) || containsHan(titleGlyphs)) {
1089|       throw new Error("encoded payload still contains han");
1090|     }
1091|
1092|     const fontFamily = `${FONT_FAMILY_PREFIX}${chapter.id.replace(/[^a-zA-Z0-9-]/g, "").slice(0, 24)
1093|   || "ch"}`;
1094|     const psName = `PrSess${chapter.id.replace(/[^a-zA-Z0-9]/g, "").slice(0, 24) || "Ch"}`;
1095|     const outputPath = path.join(generatedDir, `${chapter.id}.woff2`);
1096|
1097|     const mappings = [];
1098|     for (const [from, to] of forward) {
1099|       mappings.push({
1100|         from: from.codePointAt(0),
```

```
        to: to.codePointAt(0),
      });
    }

    await buildFont({
      masterPath: resolveMaster(),
      outputPath,
      fontFamily,
      psName,
      mappings,
      passthrough: [9, 10, 13, 32, 160, 0x3000],
    });

    const buffer = fs.readFileSync(outputPath);
    const record = { fontFamily, titleGlyphs, bodyGlyphs, buffer, revision: key };
    evictOtherRevisions(cache, chapter.id, key);
    cache.set(key, record);
    return record;
  }

  return { forChapter, masterPath };
}


// ===== FILE: packages/reader-api/src/server.js =====
import http from "node:http";
import path from "node:path";
import crypto from "node:crypto";
import { fileURLToPath } from "node:url";
import { loadChapters } from "./chapters.js";
import { loadConfig } from "./config.js";
import { createObfuscator } from "./obfuscate.js";
import { clientIp, createRateLimiter } from "./rateLimit.js";
import { fetchArticleById } from "./remoteArticle.js";
import { createTicketStore } from "./tickets.js";

const here = path.dirname(fileURLToPath(import.meta.url));

function sendJson(res, status, body, extraHeaders = {}) {
  const json = JSON.stringify(body);
  res.writeHead(status, {
    "Content-Type": "application/json; charset=utf-8",
    "Cache-Control": "no-store",
    ...extraHeaders,
  });
  res.end(json);
}

function corsHeaders(origin, allowedOrigins) {
  if (origin && allowedOrigins.includes(origin)) {
```

```
    return {
      "Access-Control-Allow-Origin": origin,
      Vary: "Origin",
      "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
      "Access-Control-Allow-Headers": "Content-Type, X-Reader-Ticket",
      "Access-Control-Max-Age": "600",
    };
  }
  return { Vary: "Origin" };
}

function originDenied(origin, allowedOrigins) {
  return Boolean(origin) && !allowedOrigins.includes(origin);
}

function readBody(req, limit = 32_768) {
  return new Promise((resolve, reject) => {
    const chunks = [];
    let size = 0;
    req.on("data", (chunk) => {
      size += chunk.length;
      if (size > limit) {
        reject(new Error("too_large"));
        req.destroy();
        return;
      }
      chunks.push(chunk);
    });
    req.on("end", () => resolve(Buffer.concat(chunks).toString("utf8")));
    req.on("error", reject);
  });
}

function parseUrl(req) {
  return new URL(req.url, "http://127.0.0.1");
}

export function createServer(overrides = {}) {
  const repoRoot =
    overrides.repoRoot ?? path.resolve(here, "../../..");
  const config = loadConfig({ ...overrides, repoRoot });
  const chapters = loadChapters(repoRoot);
  const tickets = createTicketStore({ ttlMs: config.ticketTtlMs });
  const remoteInflight = new Map();

  async function resolveChapter(chapterId) {
    if (chapters.has(chapterId)) {
      return chapters.get(chapterId);
    }
    if (!config.articleByIdUrl) {
```

```
      return null;
    }
    if (remoteInflight.has(chapterId)) {
      return remoteInflight.get(chapterId);
    }
    const pending = fetchArticleById(config.articleByIdUrl, chapterId)
      .catch(() => null)
      .finally(() => {
        remoteInflight.delete(chapterId);
      });
    remoteInflight.set(chapterId, pending);
    return pending;
  }

  const obfuscator = createObfuscator({
    repoRoot,
    generatedDir: path.join(here, "../generated-fonts"),
  });
  const allowRequest = createRateLimiter({ perMin: config.rateLimitPerMin });

  const server = http.createServer(async (req, res) => {
    const origin = req.headers.origin;
    const extra = corsHeaders(origin, config.allowedOrigins);

    try {
      if (originDenied(origin, config.allowedOrigins)) {
        sendJson(res, 403, { error: "origin_not_allowed" }, extra);
        return;
      }

      if (req.method === "OPTIONS") {
        res.writeHead(204, extra);
        res.end();
        return;
      }

      if (!allowRequest(clientIp(req))) {
        sendJson(res, 429, { error: "rate_limited" }, extra);
        return;
      }

      const url = parseUrl(req);

      if (req.method === "GET" && url.pathname === "/v1/reader/meta") {
        const list = [...chapters.values()].map((ch) => ({
          id: ch.id,
          title: ch.title,
        }));
        sendJson(
          res,
```

```
          200,
          {
            ticketTtlSec: Math.round(config.ticketTtlMs / 1000),
            siteKey: config.siteKey,
            chapters: list,
          },
          extra,
        );
        return;
      }

      if (req.method === "POST" && url.pathname === "/v1/reader/ticket") {
        let payload = {};
        try {
          const raw = await readBody(req);
          payload = raw ? JSON.parse(raw) : {};
        } catch {
          sendJson(res, 400, { error: "invalid_json" }, extra);
          return;
        }

        if (payload.siteKey && payload.siteKey !== config.siteKey) {
          sendJson(res, 403, { error: "invalid_site_key" }, extra);
          return;
        }

        const chapterId = String(payload.chapterId || "");
        const found = await resolveChapter(chapterId);
        if (!found) {
          sendJson(res, 404, { error: "chapter_not_found" }, extra);
          return;
        }

        const sessionId = crypto.randomUUID();
        const row = tickets.issue(chapterId, sessionId);
        sendJson(
          res,
          200,
          {
            ticket: row.ticket,
            sessionId: row.sessionId,
            expiresAt: new Date(row.expiresAt).toISOString(),
            chapterId: row.chapterId,
          },
          extra,
        );
        return;
      }

      if (req.method === "GET" && url.pathname === "/v1/reader/chapter") {
```

```
        const ticket =
          url.searchParams.get("ticket") ||
          req.headers["x-reader-ticket"] ||
          "";
        const row = tickets.get(String(ticket));
        if (!row) {
          sendJson(res, 401, { error: "invalid_ticket" }, extra);
          return;
        }
        const chapter = await resolveChapter(row.chapterId);
        if (!chapter) {
          sendJson(res, 404, { error: "chapter_not_found" }, extra);
          return;
        }

        let packed;
        try {
          packed = await obfuscator.forChapter(chapter);
        } catch (err) {
          console.error("font_unavailable", err && err.message ? err.message : err);
          sendJson(res, 503, { error: "font_unavailable" }, extra);
          return;
        }

        tickets.putFont(row.fontId, {
          buffer: packed.buffer,
          expiresAt: row.expiresAt,
        });

        sendJson(
          res,
          200,
          {
            chapterId: chapter.id,
            titleGlyphs: packed.titleGlyphs,
            bodyGlyphs: packed.bodyGlyphs,
            fontFamily: packed.fontFamily,
            fontUrl: `${config.publicBase}/v1/fonts/${row.fontId}.woff2`,
          },
          extra,
        );
        return;
      }

      const fontMatch = url.pathname.match(/^\/v1\/fonts\/([A-Za-z0-9_-]+)\.woff2$/);
      if (req.method === "GET" && fontMatch) {
        const font = tickets.getFont(fontMatch[1]);
        if (!font) {
          res.writeHead(404, {
            "Content-Type": "application/json; charset=utf-8",
```

```
            ...extra,
          });
          res.end(JSON.stringify({ error: "font_not_found" }));
          return;
        }
        res.writeHead(200, {
          "Content-Type": "font/woff2",
          "Cache-Control": "private, max-age=300",
          "Content-Length": font.buffer.length,
          ...extra,
        });
        res.end(font.buffer);
        return;
      }

      sendJson(res, 404, { error: "not_found" }, extra);
    } catch {
      sendJson(res, 500, { error: "internal" }, extra);
    }
  });

  server.locals = { config, chapters };
  return server;
}


// ===== FILE: packages/reader-api/src/index.js =====
import { createServer } from "./server.js";
import { loadConfig } from "./config.js";

const config = loadConfig({
  articleByIdUrl:
    process.env.READER_ARTICLE_BY_ID_URL ??
    "http://www.missimylan.info:8082/article/byId",
});
const host = process.env.READER_LISTEN_HOST ?? "127.0.0.1";
const server = createServer(config);

server.listen(config.port, host, () => {
  console.log(`reader-api http://${host}:${config.port}`);
});


// ===== FILE: packages/reader-web/src/server.js =====
import fs from "node:fs";
import http from "node:http";
import path from "node:path";
import { fileURLToPath } from "node:url";

const here = path.dirname(fileURLToPath(import.meta.url));
```

```
export const PUBLIC_DIR = path.resolve(here, "../public");
export const DEFAULT_PORT = 3981;
export const DEFAULT_API_BASE = "http://127.0.0.1:3980";

const MIME = {
  ".html": "text/html; charset=utf-8",
  ".js": "text/javascript; charset=utf-8",
  ".css": "text/css; charset=utf-8",
};

function safeJoin(root, rel) {
  const resolved = path.resolve(root, rel);
  const rootWithSep = root.endsWith(path.sep) ? root : root + path.sep;
  if (resolved !== root && !resolved.startsWith(rootWithSep)) {
    return null;
  }
  return resolved;
}

export function createServer(overrides = {}) {
  const apiBase = String(overrides.apiBase ?? process.env.READER_API_PUBLIC_BASE ?? DEFAULT_API_BASE
).replace(/\/$/, "");
  const publicDir = overrides.publicDir ?? PUBLIC_DIR;

  return http.createServer((req, res) => {
    const url = new URL(req.url || "/", "http://127.0.0.1");
    if (req.method !== "GET" && req.method !== "HEAD") {
      res.writeHead(405);
      res.end();
      return;
    }

    let rel = url.pathname === "/read" || url.pathname === "/" ? "/read.html" : url.pathname;
    const filePath = safeJoin(publicDir, rel.replace(/^\/+/, ""));
    if (!filePath || !fs.existsSync(filePath) || !fs.statSync(filePath).isFile()) {
      res.writeHead(404, { "Content-Type": "text/plain; charset=utf-8" });
      res.end("not found");
      return;
    }

    let body = fs.readFileSync(filePath);
    const ext = path.extname(filePath);
    if (ext === ".html") {
      body = Buffer.from(
        body.toString("utf8").replaceAll("__API_BASE__", apiBase),
        "utf8",
      );
    }

    res.writeHead(200, {
```

```
      "Content-Type": MIME[ext] || "application/octet-stream",
      "Cache-Control": "no-store",
      "Content-Length": body.length,
    });
    if (req.method === "HEAD") {
      res.end();
      return;
    }
    res.end(body);
  });
}


// ===== FILE: packages/reader-web/src/index.js =====
import { createServer, DEFAULT_PORT } from "./server.js";

const port = Number(process.env.READER_WEB_PORT ?? DEFAULT_PORT);
const host = process.env.READER_LISTEN_HOST ?? "127.0.0.1";
const server = createServer();

server.listen(port, host, () => {
  console.log(`reader-web http://${host}:${port}/read?chapterId=sample`);
});


// ===== FILE: packages/reader-web/public/read.html =====
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>阅读</title>
  <link rel="stylesheet" href="/read.css" />
</head>
<body data-theme="light">
  <header class="toolbar">
    <button type="button" id="size-down" aria-label="减小字号">A-</button>
    <button type="button" id="size-up" aria-label="增大字号">A+</button>
    <button type="button" id="theme-toggle" aria-label="切换深浅背景">深色</button>
  </header>
  <div class="reader-main" id="reader-main" data-wm="off">
    <div id="watermarks" aria-hidden="true">
      <div class="wm wm-light" id="wm-light"></div>
      <div class="wm wm-dark" id="wm-dark"></div>
    </div>
    <p id="status" class="status" hidden></p>
    <article id="chapter" hidden>
      <h1 id="title"></h1>
      <div id="body"></div>
    </article>
```

```
  </div>
  <script>
    window.__PETRICHOR_API__ = "__API_BASE__";
  </script>
  <script src="/read.js"></script>
</body>
</html>


// ===== FILE: packages/reader-web/public/read.css =====
:root {
  --bg: #f6f1e7;
  --fg: #1a1a1a;
  --muted: #666;
  --bar: #efe8d8;
  --wm-light: rgba(26, 26, 26, 0.028);
  --wm-dark: rgba(26, 26, 26, 0.08);
}

html[data-theme="dark"] {
  --bg: #1c1b19;
  --fg: #e8e2d6;
  --muted: #a39b8c;
  --bar: #2a2824;
  --wm-light: rgba(232, 226, 214, 0.03);
  --wm-dark: rgba(232, 226, 214, 0.1);
}

* { box-sizing: border-box; }

html, body {
  margin: 0;
  min-height: 100vh;
  background: var(--bg);
  color: var(--fg);
}

body {
  display: flex;
  flex-direction: column;
}

html.embedded,
html.embedded body {
  min-height: 0;
}

.reader-main {
  position: relative;
  flex: 1 1 auto;
```

```
}

#watermarks,
.wm {
  position: absolute;
  inset: 0;
  pointer-events: none;
  user-select: none;
  z-index: 0;
}

.wm {
  background-repeat: repeat;
}

.reader-main[data-wm="off"] .wm,
.reader-main[data-wm="light"] .wm-dark,
.reader-main[data-wm="dark"] .wm-light {
  display: none;
}

.wm-light {
  background-position: 0 0;
}

.wm-dark {
  background-position: 72px 48px;
}

#status,
#chapter {
  position: relative;
  z-index: 1;
}

.toolbar {
  display: flex;
  gap: 0.5rem;
  padding: 0.75rem 1.25rem;
  background: var(--bar);
  position: sticky;
  top: 0;
}

.toolbar button {
  font: 14px/1.2 sans-serif;
  border: 1px solid var(--muted);
  background: var(--bg);
  color: var(--fg);
  padding: 0.35rem 0.7rem;
```

```
  border-radius: 4px;
  cursor: pointer;
}

.status {
  font: 16px/1.5 sans-serif;
  padding: 2rem 1.5rem;
  color: var(--muted);
}

#chapter {
  max-width: 40rem;
  margin: 1.5rem auto 3rem;
  padding: 0 1.25rem;
  font-size: var(--reader-size, 20px);
  line-height: 1.9;
}

#title {
  font-size: 1.15em;
  font-weight: 600;
  margin: 0 0 1rem;
}

#body {
  word-break: break-word;
}

#body p,
#body blockquote,
#body h1,
#body h2,
#body h3,
#body ul,
#body ol {
  margin: 0 0 0.85em;
}

#body ul,
#body ol {
  padding-left: 1.4em;
}

#body img {
  max-width: 100%;
  height: auto;
  display: block;
  margin: 0.75em auto;
}

```

```
#body a {
  color: inherit;
  text-underline-offset: 0.15em;
}

#body .pr-align-left { text-align: left; }
#body .pr-align-center { text-align: center; }
#body .pr-align-right { text-align: right; }
#body .pr-align-justify { text-align: justify; }


// ===== FILE: packages/reader-web/public/read.js =====
const SIZE_KEY = "pr-reader-font-size";
const THEME_KEY = "pr-reader-theme";
const MIN_SIZE = 16;
const MAX_SIZE = 32;

/** Hardcoded for V1. mode: off | light | dark | both */
const WATERMARK = {
  mode: "both",
  text: "Petrichor",
};

const statusEl = document.getElementById("status");
const chapterEl = document.getElementById("chapter");
const titleEl = document.getElementById("title");
const bodyEl = document.getElementById("body");
const themeBtn = document.getElementById("theme-toggle");
const readerMainEl = document.getElementById("reader-main");
const wmLightEl = document.getElementById("wm-light");
const wmDarkEl = document.getElementById("wm-dark");

function apiBase() {
  return String(window.__PETRICHOR_API__ || "http://127.0.0.1:3980").replace(/\/$/, "");
}

const MSG_TYPE = "petrichor-reader:resize";

function inIframe() {
  return window.parent !== window;
}

function postHeight() {
  if (!inIframe()) {
    return;
  }
  const height = Math.max(
    document.documentElement.scrollHeight,
    document.body.scrollHeight,
    1,
```

```
  );
  window.parent.postMessage({ type: MSG_TYPE, height }, "*");
}

function showStatus(message) {
  statusEl.hidden = false;
  statusEl.textContent = message;
  chapterEl.hidden = true;
  postHeight();
}

function escapeXml(text) {
  return String(text)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll('"', "&quot;")
    .replaceAll("'", "&apos;");
}

function watermarkTile(text, fill, rotate) {
  const svg =
    `<svg xmlns="http://www.w3.org/2000/svg" width="280" height="200">` +
    `<text x="140" y="100" text-anchor="middle" dominant-baseline="middle"` +
    ` fill="${fill}" font-size="18" font-family="sans-serif"` +
    ` transform="rotate(${rotate} 140 100)">${escapeXml(text)}</text></svg>`;
  return `url("data:image/svg+xml,${encodeURIComponent(svg)}")`;
}

function applyWatermarks() {
  if (!readerMainEl || !wmLightEl || !wmDarkEl) {
    return;
  }
  const mode = WATERMARK.mode;
  const text = String(WATERMARK.text || "").trim();
  const enabled = text && (mode === "light" || mode === "dark" || mode === "both");
  readerMainEl.dataset.wm = enabled ? mode : "off";
  if (!enabled) {
    wmLightEl.style.backgroundImage = "none";
    wmDarkEl.style.backgroundImage = "none";
    return;
  }
  const styles = getComputedStyle(document.documentElement);
  const lightFill = styles.getPropertyValue("--wm-light").trim() || "rgba(0,0,0,0.03)";
  const darkFill = styles.getPropertyValue("--wm-dark").trim() || "rgba(0,0,0,0.08)";
  wmLightEl.style.backgroundImage = watermarkTile(text, lightFill, -28);
  wmDarkEl.style.backgroundImage = watermarkTile(text, darkFill, -28);
}

function applyChrome() {
```

```
  const size = Number(localStorage.getItem(SIZE_KEY) || 20);
  const theme = localStorage.getItem(THEME_KEY) === "dark" ? "dark" : "light";
  document.documentElement.style.setProperty("--reader-size", `${size}px`);
  document.documentElement.dataset.theme = theme;
  document.body.dataset.theme = theme;
  themeBtn.textContent = theme === "dark" ? "浅色" : "深色";
  applyWatermarks();
  postHeight();
}

function setSize(next) {
  const size = Math.min(MAX_SIZE, Math.max(MIN_SIZE, next));
  localStorage.setItem(SIZE_KEY, String(size));
  applyChrome();
}

function toggleTheme() {
  const next = document.documentElement.dataset.theme === "dark" ? "light" : "dark";
  localStorage.setItem(THEME_KEY, next);
  applyChrome();
}

async function loadFont(fontFamily, fontUrl) {
  const face = new FontFace(fontFamily, `url(${fontUrl})`, { display: "block" });
  const loaded = await face.load();
  document.fonts.add(loaded);
}

async function readChapter() {
  const chapterId = new URLSearchParams(location.search).get("chapterId") || "";
  if (!chapterId) {
    showStatus("缺少章节");
    return;
  }

  const ticketRes = await fetch(`${apiBase()}/v1/reader/ticket`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ chapterId, siteKey: "pk_dev" }),
  });
  if (!ticketRes.ok) {
    showStatus("内容无法显示");
    return;
  }
  const ticketJson = await ticketRes.json();
  const ticket = ticketJson.ticket;
  if (!ticket) {
    showStatus("内容无法显示");
    return;
  }
```

```
1801|
1802|   const chapterRes = await fetch(
1803|     `${apiBase()}/v1/reader/chapter?ticket=${encodeURIComponent(ticket)}`,
1804|   );
1805|   if (!chapterRes.ok) {
1806|     showStatus("内容无法显示");
1807|     return;
1808|   }
1809|   const payload = await chapterRes.json();
1810|   if (!payload.fontUrl || !payload.fontFamily || !payload.bodyGlyphs) {
1811|     showStatus("内容无法显示");
1812|     return;
1813|   }
1814|
1815|   try {
1816|     await loadFont(payload.fontFamily, payload.fontUrl);
1817|   } catch {
1818|     showStatus("内容无法显示");
1819|     return;
1820|   }
1821|
1822|   const family = `"${payload.fontFamily}"`;
1823|   titleEl.textContent = payload.titleGlyphs || "";
1824|   bodyEl.innerHTML = payload.bodyGlyphs;
1825|   titleEl.style.fontFamily = family;
1826|   bodyEl.style.fontFamily = family;
1827|   statusEl.hidden = true;
1828|   chapterEl.hidden = false;
1829|   postHeight();
1830| }
1831|
1832| document.getElementById("size-down").addEventListener("click", () => {
1833|   const current = Number(localStorage.getItem(SIZE_KEY) || 20);
1834|   setSize(current - 2);
1835| });
1836| document.getElementById("size-up").addEventListener("click", () => {
1837|   const current = Number(localStorage.getItem(SIZE_KEY) || 20);
1838|   setSize(current + 2);
1839| });
1840| themeBtn.addEventListener("click", toggleTheme);
1841|
1842| if (inIframe()) {
1843|   document.documentElement.classList.add("embedded");
1844| }
1845|
1846| applyChrome();
1847| readChapter().catch(() => {
1848|   showStatus("内容无法显示");
1849| });
1850|
```

```
if (typeof ResizeObserver === "function") {
  new ResizeObserver(() => postHeight()).observe(document.documentElement);
}


// ===== FILE: packages/reader-web/public/embed.js =====
(function () {
  const ATTR = "data-petrichor-reader";
  const MSG_TYPE = "petrichor-reader:resize";
  const script = document.currentScript;
  const readerOrigin = script && script.src
    ? new URL(script.src, location.href).origin
    : "http://127.0.0.1:3981";

  function chapterSrc(chapterId) {
    return `${readerOrigin}/read?chapterId=${encodeURIComponent(chapterId)}`;
  }

  function mount(el) {
    if (!el || el.nodeType !== 1) {
      return;
    }
    if (!el.hasAttribute(ATTR)) {
      return;
    }
    const chapterId = el.getAttribute("data-chapter-id") || "";
    if (!chapterId) {
      return;
    }
    let iframe = el.querySelector("iframe[data-pr-frame]");
    if (!iframe) {
      iframe = document.createElement("iframe");
      iframe.setAttribute("data-pr-frame", "1");
      iframe.setAttribute("title", "petrichor-reader");
      iframe.style.cssText =
        "width:100%;border:0;display:block;max-width:100%;overflow:hidden;height:240px;";
      el.appendChild(iframe);
    }
    if (iframe.getAttribute("data-chapter-id") !== chapterId) {
      iframe.setAttribute("data-chapter-id", chapterId);
      iframe.src = chapterSrc(chapterId);
    }
  }

  function scan(root) {
    if (!root) {
      return;
    }
    if (root.nodeType === 1 && root.hasAttribute && root.hasAttribute(ATTR)) {
      mount(root);
```

```
    }
    if (root.querySelectorAll) {
      root.querySelectorAll("[" + ATTR + "]").forEach(mount);
    }
  }

  window.addEventListener("message", function (event) {
    if (event.origin !== readerOrigin) {
      return;
    }
    const data = event.data;
    if (!data || data.type !== MSG_TYPE) {
      return;
    }
    const height = Number(data.height);
    if (!Number.isFinite(height) || height < 1) {
      return;
    }
    document.querySelectorAll("iframe[data-pr-frame]").forEach(function (frame) {
      if (frame.contentWindow === event.source) {
        frame.style.height = Math.ceil(height) + "px";
      }
    });
  });

  function boot() {
    scan(document);
    const observer = new MutationObserver(function (records) {
      for (let i = 0; i < records.length; i++) {
        const rec = records[i];
        if (rec.type === "attributes") {
          mount(rec.target);
          continue;
        }
        rec.addedNodes.forEach(scan);
        if (rec.removedNodes.length) {
          scan(rec.target);
        }
      }
    });
    observer.observe(document.documentElement, {
      childList: true,
      subtree: true,
      attributes: true,
      attributeFilter: ["data-chapter-id", "data-petrichor-reader"],
    });
  }

  if (document.readyState === "loading") {
    document.addEventListener("DOMContentLoaded", boot);
```

```
  } else {
    boot();
  }

  window.petrichorReaderScan = function () {
    scan(document);
  };
})();


// ===== FILE: examples/serve.mjs =====
import fs from "node:fs";
import http from "node:http";
import path from "node:path";
import { fileURLToPath } from "node:url";

const here = path.dirname(fileURLToPath(import.meta.url));
const port = Number(process.env.READER_HOST_PORT ?? 3982);
const hostFile = path.join(here, "host.html");

const server = http.createServer((req, res) => {
  const url = new URL(req.url || "/", "http://127.0.0.1");
  if (url.pathname !== "/" && url.pathname !== "/host.html") {
    res.writeHead(404);
    res.end("not found");
    return;
  }
  const body = fs.readFileSync(hostFile);
  res.writeHead(200, {
    "Content-Type": "text/html; charset=utf-8",
    "Cache-Control": "no-store",
    "Content-Length": body.length,
  });
  res.end(body);
});

server.listen(port, "127.0.0.1", () => {
  console.log(`example host http://127.0.0.1:${port}/host.html`);
});


// ===== FILE: examples/host.html =====
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>宿主示例（模拟外站）</title>
  <style>
    body { font-family: sans-serif; max-width: 44rem; margin: 2rem auto; line-height: 1.6; color: #2
```

```
22; }
    .meta { color: #555; }
    .comments { margin-top: 2rem; padding-top: 1rem; border-top: 1px solid #ddd; }
  </style>
</head>
<body>
  <h1>模拟博客文章页</h1>
  <p class="meta">目录标题（明文）：示例章节</p>
  <p class="meta">正文在下方 iframe 中；本页源码与 innerText 都不应出现章节原句。</p>

  <script src="http://127.0.0.1:3981/embed.js" async></script>
  <div data-petrichor-reader data-chapter-id="sample"></div>

  <section class="comments">
    <h2>评论（明文，模拟）</h2>
    <p>写得真好。</p>
  </section>
</body>
</html>


// ===== FILE: packages/font-tool/test/mapText.test.js =====
import assert from "node:assert/strict";
import test from "node:test";
import {
  buildMapping,
  decodeText,
  encodeText,
  mappingFromJson,
  mappingToJson,
} from "../src/mapText.js";
import { isPassthroughChar } from "../src/policy.js";

const SAMPLE =
  "春江潮水连海平，海上明月共潮生。\n滟滟随波千万里，何处春江无月明！";

test("whitespace is not mapped; punctuation is mapped", () => {
  assert.equal(isPassthroughChar(" "), true);
  assert.equal(isPassthroughChar("\n"), true);
  assert.equal(isPassthroughChar("  "), true);
  assert.equal(isPassthroughChar("，"), false);
  assert.equal(isPassthroughChar("春"), false);
});

test("encoded text does not contain the original sentence", () => {
  const { forward } = buildMapping(SAMPLE);
  const encoded = encodeText(SAMPLE, forward);
  assert.equal(encoded.includes("春江潮水连海平"), false);
  assert.equal(encoded.includes("海上明月共潮生"), false);
  assert.match(encoded, /\n/);
```

```
});

test("mapping is reversible inside the font-tool", () => {
  const { forward } = buildMapping(SAMPLE);
  const encoded = encodeText(SAMPLE, forward);
  assert.equal(decodeText(encoded, forward), SAMPLE);
  const roundTrip = mappingFromJson(mappingToJson(forward));
  assert.equal(decodeText(encoded, roundTrip), SAMPLE);
});


// ===== FILE: packages/font-tool/test/pipeline.test.js =====
import assert from "node:assert/strict";
import fs from "node:fs";
import path from "node:path";
import test from "node:test";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";
import { findMasterFont } from "../src/buildFont.js";
import { decodeText, mappingFromJson } from "../src/mapText.js";

const here = path.dirname(fileURLToPath(import.meta.url));
const repoRoot = path.resolve(here, "../../..");
const pkgRoot = path.resolve(here, "..");

test("pipeline: preview has no original sentence; map reverses internally", async (t) => {
  const master = findMasterFont(path.join(repoRoot, "fonts/master"));
  if (!master) {
    t.skip("master font not downloaded yet");
    return;
  }

  execFileSync(process.execPath, [path.join(pkgRoot, "src/cli.js")], {
    cwd: pkgRoot,
    stdio: "inherit",
  });

  const chapter = fs
    .readFileSync(path.join(repoRoot, "fixtures/chapters/sample.txt"), "utf8")
    .replace(/^\uFEFF/, "");
  const firstLine = chapter.trim().split(/\n/)[0];
  const preview = fs.readFileSync(
    path.join(repoRoot, "fixtures/preview.html"),
    "utf8",
  );
  const mapJson = JSON.parse(
    fs.readFileSync(path.join(pkgRoot, "output/map.json"), "utf8"),
  );
  const woff2 = fs.statSync(path.join(pkgRoot, "output/subset.woff2"));

```

```
  assert.ok(woff2.size > 1000);
  assert.equal(preview.includes(firstLine), false);
  assert.match(preview, /font-family: "pr-sess-preview"/);
  assert.equal(preview.includes("Noto"), false);
  assert.equal(preview.includes("Source"), false);

  const encodedMatch = preview.match(/<div class="chapter">([\s\S]*?)<\/div>/);
  assert.ok(encodedMatch);
  const encoded = encodedMatch[1]
    .replaceAll("&lt;", "<")
    .replaceAll("&gt;", ">")
    .replaceAll("&amp;", "&");
  const decoded = decodeText(encoded, mappingFromJson(mapJson));
  assert.equal(decoded, chapter);

  const readerWeb = path.join(repoRoot, "packages/reader-web");
  const webFiles = fs.existsSync(readerWeb)
    ? fs.readdirSync(readerWeb, { recursive: true })
    : [];
  for (const rel of webFiles) {
    const full = path.join(readerWeb, rel);
    if (!fs.statSync(full).isFile()) continue;
    const src = fs.readFileSync(full, "utf8");
    assert.equal(src.includes("decodeText"), false);
  }
});


// ===== FILE: packages/reader-api/test/richText.test.js =====
import assert from "node:assert/strict";
import test from "node:test";
import { buildMapping } from "../../font-tool/src/mapText.js";
import { containsHan } from "../src/config.js";
import {
  chapterBodyHasContent,
  collectVisibleText,
  encodeRichHtml,
  normalizeChapterBody,
} from "../src/richText.js";

test("plain text becomes paragraphs and line breaks", () => {
  const html = normalizeChapterBody("上句\n 下句\n\n 下一段");
  assert.match(html, /<p>/);
  assert.match(html, /<br>/);
  assert.equal(html.includes("<script"), false);
  assert.equal(collectVisibleText(html).includes("上句"), true);
  assert.equal(collectVisibleText(html).includes("下一段"), true);
});

test("keeps bold italic image and drops script", () => {
```

```
  const html = normalizeChapterBody(
    '<p>甲<strong>乙</strong><em>丙</em></p><p><img src="https://pic.example/a.png" alt="图题"></p>' +
      '<script>alert(1)</script><p><img src="javascript:alert(1)"></p>' +
      '<p><a href="https://example.com">链</a></p>',
  );
  assert.match(html, /<strong>乙<\/strong>/);
  assert.match(html, /<em>丙<\/em>/);
  assert.match(html, /<img src="https:\/\/pic.example\/a.png"/);
  assert.match(html, /alt="图题"/);
  assert.match(html, /<a href="https:\/\/example.com" rel="noopener noreferrer">链<\/a>/);
  assert.equal(html.includes("script"), false);
  assert.equal(html.includes("javascript"), false);
  assert.equal(chapterBodyHasContent(html), true);
});

test("encodeRichHtml only remaps text and alt", () => {
  const src = normalizeChapterBody(
    '<p>春江<strong>潮水</strong></p><img src="https://pic.example/a.png" alt="明月">',
  );
  const { forward } = buildMapping(collectVisibleText(src));
  const encoded = encodeRichHtml(src, forward);
  assert.match(encoded, /<p>/);
  assert.match(encoded, /<strong>/);
  assert.match(encoded, /<img src="https:\/\/pic.example\/a.png"/);
  assert.equal(containsHan(collectVisibleText(encoded)), false);
  assert.equal(encoded.includes("春江"), false);
  assert.equal(encoded.includes("明月"), false);
  assert.equal(encoded.includes("https://pic.example/a.png"), true);
});


// ===== FILE: packages/reader-api/test/revision.test.js =====
import assert from "node:assert/strict";
import test from "node:test";
import {
  chapterRevision,
  evictOtherRevisions,
  fontCacheKey,
  normalizeUpdatedAt,
} from "../src/revision.js";

test("chapterRevision prefers updateTime and falls back to content hash", () => {
  assert.equal(normalizeUpdatedAt(""), "");
  assert.equal(chapterRevision({ updatedAt: "2026-09-07 12:00:00", title: "a", body: "b" }), "2026-0
9-07 12:00:00");
  const a = chapterRevision({ title: "甲", body: "<p>旧</p>" });
  const b = chapterRevision({ title: "甲", body: "<p>新</p>" });
  const c = chapterRevision({ title: "甲", body: "<p>旧</p>" });
  assert.notEqual(a, b);
  assert.equal(a, c);
```

```
  assert.equal(a.length, 16);
});

test("font cache keys change when revision changes", () => {
  const first = fontCacheKey({ id: "9", updatedAt: "1" });
  const second = fontCacheKey({ id: "9", updatedAt: "2" });
  assert.notEqual(first, second);
  assert.match(first, /^9/);
});

test("evictOtherRevisions drops stale keys for the same chapter", () => {
  const cache = new Map([
    [fontCacheKey({ id: "9", updatedAt: "1" }), { old: true }],
    [fontCacheKey({ id: "8", updatedAt: "1" }), { keepOther: true }],
  ]);
  const keep = fontCacheKey({ id: "9", updatedAt: "2" });
  cache.set(keep, { fresh: true });
  evictOtherRevisions(cache, "9", keep);
  assert.equal(cache.has(fontCacheKey({ id: "9", updatedAt: "1" })), false);
  assert.equal(cache.get(keep).fresh, true);
  assert.equal(cache.get(fontCacheKey({ id: "8", updatedAt: "1" })).keepOther, true);
});


// ===== FILE: packages/reader-api/test/obfuscate.test.js =====
import assert from "node:assert/strict";
import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import test from "node:test";
import { createObfuscator } from "../src/obfuscate.js";

function tempDir() {
  return fs.mkdtempSync(path.join(os.tmpdir(), "pr-obfuscate-"));
}

test("font cache hits the same revision and rebuilds after updateTime changes", async () => {
  const generatedDir = tempDir();
  let builds = 0;
  const obfuscator = createObfuscator({
    repoRoot: generatedDir,
    generatedDir,
    masterPath: path.join(generatedDir, "master.otf"),
    buildFont: async (job) => {
      builds += 1;
      fs.writeFileSync(job.outputPath, Buffer.from("wOF2fake"));
    },
  });

  const v1 = { id: "9", title: "标题", body: "<p>旧文</p>", updatedAt: "100" };
```

```
  const first = await obfuscator.forChapter(v1);
  const second = await obfuscator.forChapter({ ...v1 });
  assert.equal(builds, 1);
  assert.equal(first.bodyGlyphs, second.bodyGlyphs);

  const v2 = { ...v1, body: "<p>新文多一字</p>", updatedAt: "200" };
  const third = await obfuscator.forChapter(v2);
  assert.equal(builds, 2);
  assert.notEqual(first.revision, third.revision);
  assert.notEqual(first.bodyGlyphs, third.bodyGlyphs);
});


// ===== FILE: packages/reader-api/test/remoteArticle.test.js =====
import assert from "node:assert/strict";
import http from "node:http";
import test from "node:test";
import { createServer } from "../src/server.js";
import { fetchArticleById, htmlToPlain } from "../src/remoteArticle.js";

test("htmlToPlain strips tags and keeps line breaks", () => {
  const out = htmlToPlain("<p>春江</p><br/>潮水 连海平");
  assert.equal(out.includes("<"), false);
  assert.match(out, /春江/);
  assert.match(out, /潮水 连海平/);
});

test("fetchArticleById keeps sanitized HTML body", async () => {
  const server = http.createServer((req, res) => {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(
      JSON.stringify({
        data: {
          id: "9",
          articleChapter: "第一章",
          articleTitle: "标题",
          articleHead: "<p><em>文首</em></p>",
          articleContent: '<p>正文<strong>加重</strong></p><p><img src="https://pic.example/x.png"></p>'
,
          articleTail: "",
          updateTime: "2026-09-07 12:00:00",
          createTime: "2026-09-01 08:00:00",
        },
      }),
    );
  });
  await new Promise((resolve) => server.listen(0, "127.0.0.1", resolve));
  const { port } = server.address();
  try {
    const article = await fetchArticleById(`http://127.0.0.1:${port}/article/byId`, "9");
```

```
    assert.equal(article.id, "9");
    assert.match(article.body, /<em>文首<\/em>/);
    assert.match(article.body, /<strong>加重<\/strong>/);
    assert.match(article.body, /<img src="https:\/\/pic.example\/x.png">/);
    assert.equal(article.updatedAt, "2026-09-07 12:00:00");
  } finally {
    await new Promise((resolve) => server.close(resolve));
  }
});

test("reader refetches remote article so later updateTime is visible", async () => {
  let fetches = 0;
  let updateTime = "1";
  const cms = http.createServer((req, res) => {
    fetches += 1;
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(
      JSON.stringify({
        data: {
          id: "99",
          articleTitle: "远程章",
          articleContent: `<p>正文${updateTime}</p>`,
          updateTime,
        },
      }),
    );
  });
  await new Promise((resolve) => cms.listen(0, "127.0.0.1", resolve));
  const articleUrl = `http://127.0.0.1:${cms.address().port}/article/byId`;
  const reader = createServer({ articleByIdUrl: articleUrl });
  await new Promise((resolve) => reader.listen(0, "127.0.0.1", resolve));
  const port = reader.address().port;

  function ticket() {
    return new Promise((resolve, reject) => {
      const req = http.request(
        {
          hostname: "127.0.0.1",
          port,
          method: "POST",
          path: "/v1/reader/ticket",
          headers: { "Content-Type": "application/json" },
        },
        (res) => {
          const chunks = [];
          res.on("data", (c) => chunks.push(c));
          res.on("end", () =>
            resolve({ status: res.statusCode, text: Buffer.concat(chunks).toString("utf8") }),
          );
        },
```

```
      );
      req.on("error", reject);
      req.write(JSON.stringify({ chapterId: "99" }));
      req.end();
    });
  }

  try {
    const first = await ticket();
    assert.equal(first.status, 200);
    updateTime = "2";
    const second = await ticket();
    assert.equal(second.status, 200);
    assert.equal(fetches, 2);
  } finally {
    await new Promise((resolve) => reader.close(resolve));
    await new Promise((resolve) => cms.close(resolve));
  }
});


// ===== FILE: packages/reader-api/test/api.test.js =====
import assert from "node:assert/strict";
import http from "node:http";
import test from "node:test";
import { containsHan } from "../src/config.js";
import { createServer } from "../src/server.js";

function listen(server) {
  return new Promise((resolve) => {
    server.listen(0, "127.0.0.1", () => resolve(server.address().port));
  });
}

function request(port, { method, path, headers = {}, body } = {}) {
  return new Promise((resolve, reject) => {
    const req = http.request(
      {
        hostname: "127.0.0.1",
        port,
        method: method || "GET",
        path,
        headers,
      },
      (res) => {
        const chunks = [];
        res.on("data", (c) => chunks.push(c));
        res.on("end", () => {
          const buf = Buffer.concat(chunks);
          resolve({
```

```
            status: res.statusCode,
            headers: res.headers,
            body: buf,
            text: buf.toString("utf8"),
          });
        });
      },
    );
    req.on("error", reject);
    if (body) {
      req.write(body);
    }
    req.end();
  });
}

async function withServer(fn, overrides = {}) {
  const server = createServer(overrides);
  const port = await listen(server);
  try {
    return await fn(port, server);
  } finally {
    await new Promise((resolve, reject) =>
      server.close((err) => (err ? reject(err) : resolve())),
    );
  }
}

const SNIPPET = "春江潮水连海平";

test("ticket + chapter JSON has no han; font is woff2", async () => {
  await withServer(async (port) => {
    const ticketRes = await request(port, {
      method: "POST",
      path: "/v1/reader/ticket",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ chapterId: "sample", siteKey: "pk_dev" }),
    });
    assert.equal(ticketRes.status, 200);
    const ticketJson = JSON.parse(ticketRes.text);
    assert.equal(containsHan(ticketRes.text), false);

    const chapterRes = await request(port, {
      path: `/v1/reader/chapter?ticket=${encodeURIComponent(ticketJson.ticket)}`,
      headers: { Origin: "http://localhost:3981" },
    });
    assert.equal(chapterRes.status, 200);
    assert.equal(chapterRes.headers["access-control-allow-origin"], "http://localhost:3981");
    assert.equal(chapterRes.text.includes(SNIPPET), false);
    assert.equal(containsHan(chapterRes.text), false);
```

```
    const chapter = JSON.parse(chapterRes.text);
    assert.ok(chapter.bodyGlyphs.length > 10);
    assert.match(chapter.bodyGlyphs, /<p>/);
    assert.ok(chapter.titleGlyphs.length > 0);
    assert.match(chapter.fontFamily, /^pr-sess-/);
    assert.equal(chapter.fontFamily.includes("Noto"), false);
    assert.equal(chapter.fontFamily.includes("Source"), false);
    assert.match(chapter.fontUrl, /\/v1\/fonts\/[a-f0-9]+\.woff2$/);

    const fontPath = new URL(chapter.fontUrl).pathname;
    const fontRes = await request(port, {
      path: fontPath,
      headers: { Origin: "http://localhost:3981" },
    });
    assert.equal(fontRes.status, 200);
    assert.equal(fontRes.headers["content-type"], "font/woff2");
    assert.ok(fontRes.body.length > 1000);
    assert.equal(fontRes.body.subarray(0, 4).toString(), "wOF2");
  });
});

test("invalid ticket fails", async () => {
  await withServer(async (port) => {
    const res = await request(port, {
      path: "/v1/reader/chapter?ticket=nope",
    });
    assert.equal(res.status, 401);
    assert.equal(containsHan(res.text), false);
  });
});

test("expired ticket fails", async () => {
  await withServer(
    async (port) => {
      const ticketRes = await request(port, {
        method: "POST",
        path: "/v1/reader/ticket",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ chapterId: "sample" }),
      });
      const ticket = JSON.parse(ticketRes.text).ticket;
      await new Promise((r) => setTimeout(r, 25));
      const res = await request(port, {
        path: `/v1/reader/chapter?ticket=${encodeURIComponent(ticket)}`,
      });
      assert.equal(res.status, 401);
    },
    { ticketTtlMs: 10 },
  );
});
```

```
2501|
2502| test("blog origin is rejected", async () => {
2503|   await withServer(async (port) => {
2504|     const res = await request(port, {
2505|       method: "POST",
2506|       path: "/v1/reader/ticket",
2507|       headers: {
2508|         "Content-Type": "application/json",
2509|         Origin: "http://localhost:8080",
2510|       },
2511|       body: JSON.stringify({ chapterId: "sample" }),
2512|     });
2513|     assert.equal(res.status, 403);
2514|     assert.equal(res.headers["access-control-allow-origin"], undefined);
2515|   });
2516| });
2517|
2518| test("meta lists fixture chapter ids", async () => {
2519|   await withServer(async (port) => {
2520|     const res = await request(port, { path: "/v1/reader/meta" });
2521|     assert.equal(res.status, 200);
2522|     const json = JSON.parse(res.text);
2523|     assert.equal(json.ticketTtlSec, 300);
2524|     assert.equal(json.siteKey, "pk_dev");
2525|     assert.equal(json.chapters[0].id, "sample");
2526|   });
2527| });
2528|
2529|
2530| // ===== FILE: packages/reader-web/test/page.test.js =====
2531| import assert from "node:assert/strict";
2532| import fs from "node:fs";
2533| import http from "node:http";
2534| import path from "node:path";
2535| import test from "node:test";
2536| import { fileURLToPath } from "node:url";
2537| import { createServer } from "../src/server.js";
2538|
2539| const here = path.dirname(fileURLToPath(import.meta.url));
2540| const publicDir = path.resolve(here, "../public");
2541|
2542| function listen(server) {
2543|   return new Promise((resolve) => {
2544|     server.listen(0, "127.0.0.1", () => resolve(server.address().port));
2545|   });
2546| }
2547|
2548| function get(port, pathName) {
2549|   return new Promise((resolve, reject) => {
2550|     http.get({ hostname: "127.0.0.1", port, path: pathName }, (res) => {
```

```
      const chunks = [];
      res.on("data", (c) => chunks.push(c));
      res.on("end", () =>
        resolve({
          status: res.statusCode,
          text: Buffer.concat(chunks).toString("utf8"),
        }),
      );
    }).on("error", reject);
  });
}

test("client assets never include decode helpers or fixture prose", () => {
  for (const name of ["read.js", "read.html", "read.css", "embed.js"]) {
    const src = fs.readFileSync(path.join(publicDir, name), "utf8");
    assert.equal(src.includes("decodeText"), false);
    assert.equal(src.includes("春江潮水连海平"), false);
    assert.equal(src.includes("map.json"), false);
  }
  const html = fs.readFileSync(path.join(publicDir, "read.html"), "utf8");
  assert.match(html, /id="wm-light"/);
  assert.match(html, /id="wm-dark"/);
  const css = fs.readFileSync(path.join(publicDir, "read.css"), "utf8");
  assert.match(css, /--wm-light/);
  assert.match(css, /--wm-dark/);
  const js = fs.readFileSync(path.join(publicDir, "read.js"), "utf8");
  assert.match(js, /内容无法显示/);
  assert.match(js, /FontFace/);
  assert.match(js, /innerHTML/);
  assert.match(js, /petrichor-reader:resize/);
  assert.match(js, /mode: "both"/);
  assert.match(js, /off \| light \| dark \| both/);
  assert.equal(js.includes("console.log"), false);
  const embed = fs.readFileSync(path.join(publicDir, "embed.js"), "utf8");
  assert.match(embed, /data-petrichor-reader/);
  assert.match(embed, /MutationObserver/);
});

test("GET /read injects api base and does not embed chapter prose", async () => {
  const server = createServer({ apiBase: "http://127.0.0.1:3980" });
  const port = await listen(server);
  try {
    const res = await get(port, "/read?chapterId=sample");
    assert.equal(res.status, 200);
    assert.match(res.text, /window\.__PETRICHOR_API__ = "http:\/\/127\.0\.0\.1:3980"/);
    assert.equal(res.text.includes("春江潮水连海平"), false);
    assert.match(res.text, /src="\/read\.js"/);
  } finally {
    await new Promise((resolve, reject) =>
      server.close((err) => (err ? reject(err) : resolve())),
```

```
    );
  }
});

test("GET /embed.js is served", async () => {
  const server = createServer({ apiBase: "http://127.0.0.1:3980" });
  const port = await listen(server);
  try {
    const res = await get(port, "/embed.js");
    assert.equal(res.status, 200);
    assert.match(res.text, /data-petrichor-reader/);
  } finally {
    await new Promise((resolve, reject) =>
      server.close((err) => (err ? reject(err) : resolve())),
    );
  }
});


```